

6교시: Alert Preview

W4D3 Lesson 6 alert preview : 알람은 어떻게 만들어지고, 어떻게 전달되고, 어떻게 운영할까?

Prometheus + Alertmanager 기반 알람의 핵심 흐름과 실습 전 미리보기







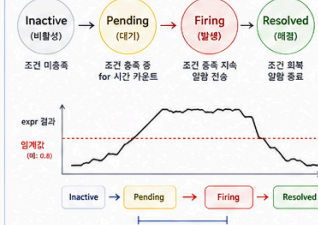
1. Prometheus 작성 (알람 규칙)

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: pod-alerts
  namespace: monitoring
spec:
  groups:
    - name: pod.rules
      rules:
        - alert: PodHighCPU
          expr: sum(rate(container_cpu_usage_seconds_total{pod=~".*", namespace="kube-*"}[5m])) > 0.8
          for: 5m
          labels:
            severity: warning
            team: platform
          annotations:
            summary: "Pod CPU 사용률 높음"
            description: "Pod CPU 사용률이 80% 이상 증가했습니다."
            value: "Pod CPU 사용률이 80% 이상 증가했습니다."
            value: "Pod CPU 사용률이 80% 이상 증가했습니다."
```

필드	설명
expr	알람 조건 (PromQL)
for	조건 지속 시간 (Pending → Firing)
labels	알람 라벨 (라우팅/그룹핑)
annotations	알람 상세 설명 (알림 메시지에 포함)

💡 너무 민감한 조건은 알람 폭주(노이즈)를 유발합니다. 적절한 임계값과 for 시간을 설정하세요.

2. 알람 상태 전이(State Transition)



expr 결과
임계값 (예: 0.8)
for: 5m

3. Alertmanager 처리 흐름



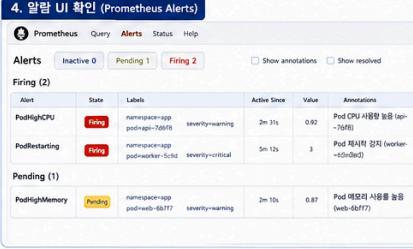
Silence (알람 억제)

- 특정 시간 동안 알람을 일시적으로 억제
- 대형: 라벨(예: alertname=PodHighCPU)
- 기간: 1h / 6h / 24h
- 유지보수/점검 중 불필요한 알람 차단

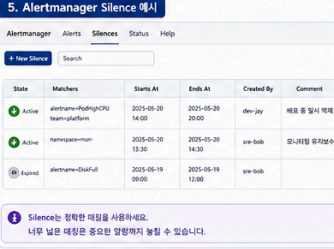
Alertmanager 주요 파일

```
route:
  group_by: ['alertname', 'namespace']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 3h
  receivers: 'default-slack'
receivers:
  - name: 'default-slack'
    slack_configs:
      - channel: '#alerts'
        send_resolved: true
```

4. 알람 UI 확인 (Prometheus Alerts)



5. Alertmanager Silence 예시



6. 알람 노이즈 vs. 실행 가능한 알람

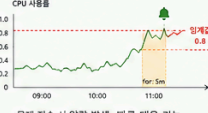
알람 노이즈 (Bad Example)

- 임계값이 너무 낮음
- for 시간 없음
- 너무 많은 알람 동시 발생



실행 가능한 알람 (Good Example)

- 적절한 임계값
- for 시간 설정 (예: 5m)
- 라벨/주석으로 맥락 제공



7. 이미지 빌드 & Docker Hub Push (가이드)



8. 이미지 Pull & 실행 (검증)

```
$ docker login
Username: dev-jay
Password: *****
Login Succeeded

$ docker pull dev-jay/sample-app:v1.2.0
v1.2.0: Pulling from dev-jay/sample-app
Digest: sha256:9f9f...
Status: Downloaded newer image for dev-jay/sample-app:v1.2.0

$ docker ps
CONTAINER ID   IMAGE          STATUS
7f9e1c2d3d0a   dev-jay/sample-app:v1.2.0   Up 5 minutes
```

9. 기타 도구 대안 비교

구분	GitHub Actions	Jenkins	TeamCity	AWS CodePipeline
관리 주체	GitHub	Self-Hosted	Self-Hosted	AWS
설정 난이도	쉬움	보통~어려움	보통	보통
확장성	우수	높음	높음	높음
비용	무료~	인프라 비용	라이선스 필요	사용량 기반 요금
GitHub 통합	최적	플러그인	연동 가능	연동 가능

알람은 노이즈가 아니라 문제의 신호! 설정이 결과를 좌우함.

💡 Tip: 작은 변경 → 빠른 피드백 → 안전한 배포
관속성과 자동화가 선택할 수 있는 운영의 핵심입니다.

수업 목표

- alert rule, pending, firing, resolved, silence의 의미를 구분한다.
- threshold를 너무 낮게 잡으면 alert noise가 생긴다는 점을 이해한다.
- PrometheusRule manifest를 preview 수준으로 적용한다.

Alert의 기본 흐름

PromQL 조건

- > 일정 시간 유지 (for)
- > pending
- > firing
- > Alertmanager routing
- > silence 또는 notification

alert는 “metric이 있다”에서 끝나지 않고, 사람이 개입해야 할 조건을 정의하는 것이다.

file:///C:/Users/Public/Documents/ESTsoft/CreatorTemp/lecture-pdf-xn61acbp/index.html

1/5

PrometheusRule 적용

```
kubectl apply -f week4/day3/labs/observability-scenarios/prometheus-rule-preview.yaml
kubectl -n week4-observe get prometheusrule
```

rule 일부:

```
expr: increase(kube_pod_container_status_restarts_total{namespace="week4-observe"}
[5m]) > 0
for: 1m
labels:
  severity: warning
```

해석:

필드	의미
expr	alert 조건
for	조건이 유지되어야 하는 시간
severity	routing과 우선순위 기준
annotations	사람이 읽을 설명

UI에서 확인

Prometheus:

```
http://localhost:9090/alerts
```

상태:

상태	의미
inactive	조건 불만족
pending	조건 만족 중이지만 for 대기
firing	조건이 유지되어 alert 발생

API로도 확인할 수 있다.

```
curl -s --get 'http://localhost:9090/api/v1/query' \
  --data-urlencode 'query=ALERTS{alertname="Week4ObservePodRestarting"}'
```

실제 검증 예시:

```

alertname="Week4ObservePodRestarting"
alertstate="firing"
namespace="week4-observe"
pod="crashloop-demo-..."
value="1"

```

rule 목록에서 확인:

```

Week4ObservePodRestarting firing
increase(kube_pod_container_status_restarts_total{namespace="week4-observe"}[5m]) > 0

```

pending이 필요한 이유

for가 없으면 순간적인 spike에도 alert가 바로 firing된다.

```
for: 1m
```

의미:

조건이 1분 동안 계속 참이어야 firing으로 전환

예를 들어 Pod가 배포 중 한 번 restart한 것은 정상적인 운영 이벤트일 수 있다. 하지만 짧은 시간에 반복 restart가 이어지면 사람이 봐야 할 가능성이 커진다.

Alertmanager에서 보는 것

Alertmanager는 alert를 그대로 보내는 도구가 아니라 묶고, 라우팅하고, 잠시 조용히 만드는 도구다.

기능	의미
grouping	비슷한 alert를 묶음
routing	severity/team/service에 따라 전달
inhibition	상위 장애가 있을 때 하위 alert 억제
silence	특정 조건 alert를 일정 시간 숨김

오늘은 알림 채널 연동까지 하지 않고 개념과 UI만 확인한다.

Silence preview

Alertmanager는 alert를 묶고 조용히 만드는 silence 기능을 제공한다.

접속:

```
kubectl -n monitoring port-forward svc/kube-prometheus-stack-alertmanager 9093:9093
```

브라우저:

```
http://localhost:9093
```

silence는 장애를 고치는 기능이 아니다. 이미 알고 있는 작업이나 테스트로 인해 알림이 불필요할 때 일정 시간 알림을 줄이는 기능이다.

Alert noise

나쁜 alert는 팀의 집중력을 깎는다.

나쁜 alert	문제
CPU 50% 1분	너무 흔함
Pod restart 1회	배포 중 정상일 수도 있음
target down 즉시	일시적 재시작에도 울림

더 나은 alert:

조건	이유
10분 이상 지속	일시적 spike 제거
사용자 영향 metric과 결합	실제 장애 가능성 증가
severity 구분	대응 우선순위 결정
runbook link 포함	다음 행동 명확화

alert 문장 작성 기준

좋은 alert annotation은 다음을 포함한다.

항목	예시
무엇이	week4-observe namespace Pod
어떻게	container restart 증가
얼마나	5분 동안 1회 이상
다음 행동	logs --previous와 describe pod 확인

나쁜 문장:

```
PodRestarting
```

더 나은 문장:

week4-observe namespace에서 최근 5분 동안 container restart가 증가했습니다. rollout, logs --previous, describe pod event를 확인하세요.

수업에서 이 alert가 좋은 이유:

기준	설명
대상이 좁음	week4-observe namespace만 본다
재현 가능	crashloop-demo로 즉시 조건을 만들 수 있다
다음 행동이 있음	logs --previous, describe pod로 이어진다
noise 토론 가능	restart 1회 alert가 운영에서 너무 민감할 수 있음을 설명할 수 있다

alert를 만들지 않아도 되는 경우

상황	이유
실습용 장애	사람이 이미 알고 있음
순간 spike	자동 복구 가능
영향 없는 dev namespace	noise 가능성
dashboard로 충분한 지표	즉시 개입 필요 없음

alert는 많을수록 좋은 것이 아니라 적절할수록 좋다.

Evidence Note

```
# W4D3S6 Alert preview
- PrometheusRule 이름 :
- alert expr:
- for:
- pending/firing 상태 :
- noise가 될 수 있는 이유:
- silence가 필요한 상황:
```

한 줄 요약

좋은 alert는 metric 조건이 아니라 사람이 지금 해야 할 행동과 연결되어야 한다.